เอกสารข้อกำหนดความต้องการของระบบ (Product Requirements Document) และสถาปัตยกรรมซอฟต์แวร์สำหรับระบบจัดการคำสั่งซื้ออีคอมเมิร์ซ (OMS)

1. บทนำและบริบททางธุรกิจ (Introduction and Business Context)

ในยุคดิจิทัลที่การพาณิชย์อิเล็กทรอนิกส์ (E-commerce) มีการเติบโตอย่างก้าวกระโดด ธุรกิจขนาดเล็กมีความจำเป็นอย่างยิ่งยวดที่จะต้องมีโครงสร้างพื้นฐานทางเทคโนโลยีที่สามารถรองรับการขยายตัวและจัดการกับการดำเนินงานประจำวันได้อย่างมีประสิทธิภาพ [1] หนึ่งในองค์ประกอบที่สำคัญที่สุดคือระบบจัดการคำสั่งซื้อ (Order Management System - OMS) ซึ่งทำหน้าที่เป็นศูนย์กลางในการควบคุมวงจรชีวิตของคำสั่งซื้อ ตั้งแต่การเลือกสินค้า การยืนยันคำสั่งซื้อ ไปจนถึงการจัดการสถานะโดยผู้ดูแลระบบ [2]

รายงานฉบับนี้ทำหน้าที่เป็นเอกสารข้อกำหนดความต้องการของผลิตภัณฑ์ (Product Requirements Document - PRD) แบบครบวงจร สำหรับการพัฒนาระบบ Backend และ Frontend ที่ถูกหลอมรวมเข้าด้วยกัน เพื่อรองรับฟังก์ชันการทำงานหลักของร้านค้าอีคอมเมิร์ซขนาดเล็กที่ต้องการความรวดเร็วและประสิทธิภาพสูง [4] ข้อกำหนดของระบบได้ถูกตีกรอบให้ใช้เทคโนโลยี PHP 7 ในรูปแบบสถาปัตยกรรมแบบประมวลผลฝั่งเซิร์ฟเวอร์ (Server-Side) อย่างเต็มรูปแบบ โดยไม่มีการแบ่งแยกส่วนหน้าบ้าน (Frontend) และหลังบ้าน (Backend) ผ่าน Application Programming Interface (API) และในระยะการพัฒนานี้ จะละเว้นข้อกังวลด้านความปลอดภัยของโค้ด (Code Security) เพื่อมุ่งเน้นไปที่ความสมบูรณ์ของตรรกะทางธุรกิจ (Business Logic) และการไหลเวียนของข้อมูล (Data Flow) เป็นหลัก [6]

นอกจากนี้ การออกแบบส่วนติดต่อผู้ใช้ (User Interface - UI) และประสบการณ์ผู้ใช้ (User Experience - UX) ได้ถูกกำหนดให้สะท้อนตามรูปแบบที่ระบุไว้ในภาพตัวอย่างอย่างเคร่งครัด ซึ่งประกอบไปด้วยการจัดวางรายการสินค้า (Product Grid) แถบสรุปคำสั่งซื้อ (Cart Summary) และกลไกการเลือกสินค้าที่ตอบสนองต่อผู้ใช้ได้อย่างเป็นธรรมชาติ การวิเคราะห์และข้อกำหนดทั้งหมดในรายงานฉบับนี้ จะครอบคลุมทั้งมิติของฐานข้อมูล สถาปัตยกรรม ลอจิกการทำงาน 8 ประการ และแนวทางการออกแบบ UI/UX ที่สอดคล้องกับมาตรฐานอุตสาหกรรม [8]

2. ภาพรวมของสถาปัตยกรรมระบบ (System Architecture Overview)

การตัดสินใจทางสถาปัตยกรรมสำหรับโครงการนี้มุ่งเน้นไปที่ความเรียบง่ายและวงจรการพัฒนาที่รวดเร็ว (Rapid Development Cycle) ดังนั้น สถาปัตยกรรมแบบรวมศูนย์ (Monolithic Architecture) จึงถูกนำมาใช้ [10]

2.1 สถาปัตยกรรม Monolithic ด้วย PHP 7

ในสถาปัตยกรรมนี้ โค้ดทั้งหมดตั้งแต่การเรนเดอร์ HTML, การตรวจสอบสิทธิ์ (Authentication), ตรรกะทางธุรกิจ (Business Logic), และการเข้าถึงฐานข้อมูล (Data Access) จะถูกบรรจุอยู่ในฐานรหัส (Codebase) เดียวกันและประมวลผลบนหน่วยความจำของเซิร์ฟเวอร์เดียวกัน [6] แม้ว่าเทรนด์ปัจจุบันจะมุ่งไปสู่ Microservices แต่สำหรับทีมขนาดเล็กและโปรเจกต์ที่ต้องการความรวดเร็ว Monolith นำเสนอข้อได้เปรียบที่ชัดเจนในการลดความซับซ้อนของการปรับใช้ (Deployment) และการแก้ไขข้อบกพร่อง (Debugging) [10]

โครงสร้างไดเรกทอรี (Directory Structure) จะถูกจัดระเบียบตามหลักการ Separation of Concerns แม้จะไม่มีการใช้ Framework เพื่อให้โค้ดสามารถดูแลรักษาได้ง่าย [12]:

- config.php: สำหรับการตั้งค่าการเชื่อมต่อฐานข้อมูล MySQL
- index.php: หน้าหลักสำหรับผู้ใช้ในการเลือกสินค้า (สอดคล้องกับภาพ UI)
- admin.php: หน้าแดชบอร์ดสำหรับผู้ดูแลระบบในการจัดการคำสั่งซื้อ
- actions/: โฟลเดอร์เก็บสคริปต์ PHP สำหรับประมวลผลฟอร์ม (เช่น login_action.php, add_order_action.php)
- assets/: เก็บไฟล์ CSS เพื่อจัดแต่งหน้าตา UI ให้ตรงตามภาพตัวอย่าง

การใช้ PHP 7 ซึ่งเป็นเวอร์ชันที่มีเสถียรภาพและยังมีสัดส่วนการใช้งานในตลาดสูง จะช่วยให้ระบบสามารถประมวลผลคำสั่งได้อย่างรวดเร็วผ่านกลไก OPcache แม้กระบวนการทั้งหมดจะเกิดขึ้นบนฝั่งเซิร์ฟเวอร์ [14]

3. การออกแบบฐานข้อมูลเชิงสัมพันธ์ (Database Schema Design)

เพื่อรองรับฟังก์ชันการทำงานที่ซับซ้อน เช่น การสั่งซื้อสินค้าหลายรายการในหนึ่งคำสั่งซื้อ และการมีสถานะอ้างอิง ระบบจึงต้องได้รับการออกแบบฐานข้อมูลแบบเชิงสัมพันธ์ (Relational Database) โดยยึดหลักการทำบรรทัดฐาน (Normalization) เพื่อลดความซ้ำซ้อนของข้อมูล [15]

ตารางข้อมูลและโครงสร้างได้รับการออกแบบดังต่อไปนี้:

3.1 ตาราง Users (ผู้ใช้งาน)

ใช้สำหรับจัดเก็บข้อมูลบัญชีผู้ใช้ ทั้งลูกค้าและผู้ดูแลระบบ ตามข้อกำหนด ฟิลด์ Email จะถูกตั้งค่าเป็น Unique แต่ไม่ได้เป็น Primary Key เพื่อให้มีความยืดหยุ่นในการอ้างอิงผ่าน user_id [17]

ชื่อฟิลด์ (Column)	**ชนิดข้อมูล (Type)**	**ข้อจำกัด (Constraints)**	**คำอธิบาย**
user_id	INT	PRIMARY KEY, AUTO_INCREMENT	รหัสอ้างอิงผู้ใช้งานภายในระบบ

email	VARCHAR(255)	UNIQUE, NOT NULL	อีเมลที่ใช้เป็น User Account ห้ามซ้ำกัน
first_name	VARCHAR(100)	NOT NULL	ชื่อจริงผู้ใช้งาน
last_name	VARCHAR(100)	NOT NULL	นามสกุลผู้ใช้งาน
phone	VARCHAR(20)	NOT NULL	เบอร์โทรศัพท์สำหรับติดต่อ
password	VARCHAR(255)	NOT NULL	รหัสผ่านผู้ใช้งาน
role	VARCHAR(20)	DEFAULT 'user'	ประเภทผู้ใช้ (เช่น 'user', 'admin')

3.2 ตาราง Products (สินค้า)

ใช้สำหรับจัดเก็บข้อมูลแคตตาล็อกสินค้า ซึ่งข้อมูลเหล่านี้จะถูกนำไปแสดงผลบน UI ตามภาพตัวอย่าง [18]

ชื่อฟิลด์ (Column)	**ชนิดข้อมูล (Type)**	**ข้อจำกัด (Constraints)**	**คำอธิบาย**
product_number	VARCHAR(50)	PRIMARY KEY	รหัส SKU ของสินค้า (เช่น SKU-001)
name	VARCHAR(255)	NOT NULL	ชื่อสินค้า (เช่น แล็บท็อป Asus VivoBook 15)

price	DECIMAL(10,2)	NOT NULL	ราคาสินค้า
stock_quantity	INT	NOT NULL	จำนวนสินค้าคงเหลือ

3.3 ตาราง Status_Reference (สถานะอ้างอิง)

การแยกสถานะเป็นตารางอ้างอิง (Reference Table) เป็นแนวทางปฏิบัติที่ดี (Best Practice) ที่ช่วยให้ระบบมีความยืดหยุ่นในการขยายตัวในอนาคต หากมีการเพิ่มสถานะการจัดส่งใหม่ [19]

ชื่อฟิลด์ (Column)	**ชนิดข้อมูล (Type)**	**ข้อจำกัด (Constraints)**	**คำอธิบาย**
status_id	INT	PRIMARY KEY, AUTO_INCREMENT	รหัสสถานะ
status_name	VARCHAR(100)	UNIQUE, NOT NULL	ชื่อสถานะ ('รอยืนยันคำสั่งซื้อ', 'ยืนยันคำสั่งซื้อ')

3.4 ตาราง Orders (คำสั่งซื้อหลัก)

เป็นตารางหลักที่เก็บข้อมูลส่วนหัวของคำสั่งซื้อ โดยอ้างอิงกับผู้ใช้ที่ทำรายการและสถานะปัจจุบันของคำสั่งซื้อ [8]

ชื่อฟิลด์ (Column)	**ชนิดข้อมูล (Type)**	**ข้อจำกัด (Constraints)**	**คำอธิบาย**
order_number	VARCHAR(50)	PRIMARY KEY	เลขคำสั่งซื้อสินค้า (OrderNumber)

user_email	VARCHAR(255)	FOREIGN KEY	อ้างอิงอีเมลผู้ใช้งาน (อิงตามข้อกำหนด)
status_id	INT	FOREIGN KEY	อ้างอิงตาราง Status_Reference
shipping_address	TEXT	NULLABLE	ที่อยู่จัดส่ง ซึ่งจะถูกอัปเดตภายหลังเมื่อผู้ใช้ยืนยัน
order_date	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	วันที่สร้างคำสั่งซื้อ

3.5 ตาราง Order_Details (รายละเอียดคำสั่งซื้อ)

เพื่อรองรับความสามารถในการเลือกสินค้าได้หลายอย่างในคำสั่งซื้อเดียว และแต่ละอย่างมีจำนวนแตกต่างกัน ตารางนี้จึงทำหน้าที่เป็น Bridge Table ระหว่าง Orders และ Products [16]

ชื่อฟิลด์ (Column)	**ชนิดข้อมูล (Type)**	**ข้อจำกัด (Constraints)**	**คำอธิบาย**
detail_id	INT	PRIMARY KEY, AUTO_INCREMENT	รหัสรายละเอียด (Primary Key)
order_number	VARCHAR(50)	FOREIGN KEY	อ้างอิงเลขคำสั่งซื้อจากตาราง Orders
product_number	VARCHAR(50)	FOREIGN KEY	อ้างอิงรหัสสินค้าจากตาราง Products

quantity	INT	NOT NULL	จำนวน item ของสินค้านั้นๆ ในคำสั่งซื้อ

4. ข้อกำหนดฟังก์ชันการทำงานเชิงลึก (Functional Specifications)

ฟังก์ชันการทำงานทั้ง 8 ประการได้รับการวิเคราะห์และแปลงเป็นตรรกะทางเทคนิค (Technical Logic) อย่างละเอียด ดังนี้:

ฟังก์ชันที่ 1: Seed Data Product Item และ Status Reference แบบ Auto

ระบบต้องการกลไกอัตโนมัติเพื่อตรวจสอบและสร้างข้อมูลเริ่มต้น (Seed Data) หากพบว่าฐานข้อมูลว่างเปล่า (Data Collection is empty) กลไกนี้มีความสำคัญในการลดภาระการตั้งค่าระบบในช่วงเริ่มต้น [24]

ตรรกะการประมวลผล (Processing Logic): เมื่อมีการโหลดหน้าเว็บครั้งแรก ไฟล์ PHP จะทำงานโดยตรวจสอบจำนวนแถวข้อมูลในตาราง Products และ Status_Reference ผ่านคำสั่ง SQL SELECT COUNT(*) FROM table_name หรือใช้ฟังก์ชันเช็คค่าว่างร่วมกับผลลัพธ์ของ mysqli_num_rows() [26] หากพบว่าค่าเป็น 0 (ว่างเปล่า) สคริปต์จะทำการ Execute คำสั่ง INSERT ทันที:

1. **Seed Status Reference:** สร้างเรกคอร์ดสถานะ 2 รายการ ได้แก่ {1, 'รอยืนยันคำสั่งซื้อ'} และ {2, 'ยืนยันคำสั่งซื้อ'}
2. **Seed Product Items:** ระบบจะอ้างอิงข้อมูลตั้งต้นให้สอดคล้องกับภาพตัวอย่าง UI โดยสมบูรณ์ ดังนี้:
   - SKU-001: แล็บท็อป Asus VivoBook 15 (ราคา 18,900, จำนวน 13)
   - SKU-002: เมาส์ Logitech MX Master 3 (ราคา 3,200, จำนวน 48)
   - SKU-003: คีย์บอร์ด Keychron K2 Wireless (ราคา 2,800, จำนวน 30)
   - SKU-004: จอมอนิเตอร์ Dell 24" FHD (ราคา 5,500, จำนวน 12)
   - SKU-005: หูฟัง Sony WH-1000XM5 (ราคา 9,800, จำนวน 20)
   - SKU-006: เว็บแคม Logitech C920 (ราคา 2,100, จำนวน 0 - สินค้าหมด)
   - SKU-007: SSD Samsung 1TB NVMe (ราคา 2,900, จำนวน 45)
   - SKU-008: กระเป๋าโน้ตบุ๊ค 15.6" (ราคา 850, จำนวน 60)

ฟังก์ชันที่ 2: สร้างบัญชีผู้ใช้งาน (Create User Account)

ผู้ใช้งานสามารถลงทะเบียนได้โดยระบบจะบังคับกรอกข้อมูลที่จำเป็นทั้งหมดตามข้อกำหนด การออกแบบในส่วนนี้มุ่งเน้นไปที่การลดแรงเสียดทาน (Friction) ในการกรอกฟอร์ม [28]

กระบวนการฝั่งเซิร์ฟเวอร์ (Server-side Workflow):

1. รับข้อมูลจากฟอร์มผ่านตัวแปร $_POST (Email, First Name, Last Name, Phone, Password, ConfirmPassword)
2. ตรวจสอบความถูกต้องของรหัสผ่านว่า Password ตรงกับ ConfirmPassword หรือไม่
3. ดำเนินการค้นหาอีเมลในฐานข้อมูล SELECT email FROM Users WHERE email =? เพื่อให้แน่ใจว่า

อีเมลไม่ซ้ำ (Unique check)
4. หากไม่ซ้ำ ดำเนินการ INSERT INTO Users
หมายเหตุ: ภายใต้เงื่อนไขที่ระบุว่า "ไม่ต้องสนใจเรื่องความปลอดภัยของโค้ด" รหัสผ่านจะถูกบันทึกในรูปแบบข้อความธรรมดา (Plain Text) และไม่มีการป้องกัน SQL Injection ในระยะนี้

ฟังก์ชันที่ 3: การเข้าสู่ระบบผู้ใช้งาน (User Login)

กลไกการเข้าสู่ระบบต้องมีความเรียบง่าย ผู้ใช้ทำการล็อกอินด้วย User(Email) และ Password

กระบวนการฝั่งเซิร์ฟเวอร์: เมื่อรับข้อมูล $_POST ระบบจะประมวลผลคำสั่ง SELECT * FROM Users WHERE email =? AND password =? หากพบเรกคอร์ดที่ตรงกัน ระบบจะสร้างเซสชัน (Session) ใน PHP โดยใช้คำสั่ง session_start() และเก็บค่าตัวแปรเช่น $_SESSION['user_email'] เพื่อรักษาการยืนยันตัวตนของผู้ใช้ (Authentication State) ระหว่างการเปลี่ยนหน้าเว็บ [17]

ฟังก์ชันที่ 4: การจัดการสินค้าของ Admin และการค้นหาคำสั่งซื้อ

มุมมองนี้คือหน้าปัด (Dashboard) หลักสำหรับผู้ดูแลระบบ การวิเคราะห์สถาปัตยกรรมข้อมูลชี้ให้เห็นว่า ผู้ดูแลระบบจำเป็นต้องสามารถเข้าถึงและประมวลผลข้อมูลคำสั่งซื้อได้อย่างรวดเร็ว [8]

ความสามารถในการค้นหา (Search Specification):

ผู้ดูแลระบบสามารถกรอกข้อมูลลงในแถบค้นหา (Search Bar) ระบบจะดำเนินการค้นหาแบบยืดหยุ่น (Wildcard Search) จากฟิลด์ order_number ในตาราง Orders หรือฟิลด์ first_name, last_name ในตาราง Users ผ่านการ JOIN ตาราง

การประมวลผลข้อมูลคืนค่า (Data Return):

โครงสร้างของข้อมูลที่คืนค่ากลับมาจะมีความซับซ้อนแบบลำดับชั้น (Hierarchical Data) ระบบใช้ PHP เพื่อประมวลผลผลลัพธ์ SQL โดยจัดกลุ่มข้อมูล:

1. **ข้อมูลระดับคำสั่งซื้อ (Order Level):** เลขคำสั่งซื้อ, ชื่อ-นามสกุลลูกค้า, สถานะปัจจุบัน ('รอยืนยันคำสั่งซื้อ' หรือ 'ยืนยันคำสั่งซื้อ')
2. **ข้อมูลระดับรายละเอียด (Order Detail Level):** ภายใต้แต่ละ Order จะมีชุดของรายการสินค้า (Product List) ที่เชื่อมโยงอยู่ ประกอบด้วย Product Number, ชื่อสินค้า, จำนวนที่สั่ง และราคารวม การจัดโครงสร้างแบบนี้ทำให้แอดมินสามารถมองเห็นบริบทของคำสั่งซื้อได้อย่างสมบูรณ์ในหน้าเดียว [16]

ฟังก์ชันที่ 5: การเพิ่มคำสั่งซื้อสินค้า (Add Order)

นี่คือฟังก์ชันแกนหลักที่ทำหน้าที่รับข้อมูลจากตะกร้าสินค้าในหน้าเว็บ แล้วแปลงสภาพไปสู่การจัดเก็บทางฐานข้อมูล โดยรองรับการสั่งซื้อสินค้าหลายประเภทในครั้งเดียว [30]

กลไกทางเทคนิค:

1. ระบบสร้าง OrderNumber ใหม่ (ตัวอย่าง: ใช้ฟังก์ชันผสมระหว่าง Timestamp และสุ่มตัวเลข เช่น ORD-20260222-1234)
2. บันทึกข้อมูลตั้งต้นลงตาราง Orders ได้แก่ order_number, user_email (ดึงจาก Session) และกำหนด status_id เริ่มต้นเป็น 1 ('รอยืนยันคำสั่งซื้อ')

3. วนลูป (Loop) ทั่วทุกรายการในตะกร้า (Array ของ Data ที่ต้อง Add Order Detail) ซึ่งแต่ละรายการประกอบด้วย product_number และ quantity
4. บันทึกข้อมูลแต่ละรายการลงในตาราง Order_Details
5. เมื่อกระบวนการเสร็จสิ้น ระบบจะทำคำสั่ง Return ค่า OrderNumber กลับไปยังหน้าเว็บเพื่อแจ้งให้ผู้ใช้ทราบว่าสร้างคำสั่งซื้อสำเร็จ

ฟังก์ชันที่ 6: การแก้ไขคำสั่งซื้อสินค้า (Edit Order)

ผู้ใช้สามารถเปลี่ยนแปลงรายการสินค้าภายในคำสั่งซื้อได้ตราบใดที่สถานะยังเอื้ออำนวย การแก้ไขนี้ต้องการพารามิเตอร์ 3 ส่วนคือ OrderNumber, Product Number และ Quantity (จำนวนใหม่)

กระบวนการฝั่งเซิร์ฟเวอร์:

PHP จะรับพารามิเตอร์ทั้งสามและประมวลผลคำสั่ง UPDATE Order_Details SET quantity =? WHERE order_number =? AND product_number =?

หากการแก้ไขเป็นการปรับจำนวนให้เป็น 0 (ศูนย์) ตรรกะทางธุรกิจควรดำเนินการอัปเกรดคำสั่งเป็น DELETE FROM Order_Details สำหรับรายการสินค้านั้นๆ แทน เพื่อรักษาสุขอนามัยของข้อมูล (Data Hygiene)

ฟังก์ชันที่ 7: ผู้ใช้ยืนยันสั่งซื้อสินค้า (User Confirm Order)

ฟังก์ชันนี้ทำหน้าที่เสมือนการกรอกที่อยู่สำหรับเตรียมจัดส่ง ระบบถูกระบุให้จำลองพฤติกรรมการเรียกใช้งานแบบ PUT (เนื่องจากเป็น Server-side Web App สถาปัตยกรรมของ HTML Form จะใช้เมธอด POST และส่งพารามิเตอร์ซ่อนเพื่อจำลองคำสั่ง PUT)

กลไกทางเทคนิค: ระบบนำค่า OrderNumber และข้อมูล shipping_address ใหม่ ประมวลผลผ่านคำสั่ง: UPDATE Orders SET shipping_address =? WHERE order_number =? การฝังข้อมูลที่อยู่ลงในระดับคำสั่งซื้อโดยตรงนี้ เป็นความจำเป็นเชิงสถาปัตยกรรม [21] เพื่อให้แน่ใจว่าประวัติการส่งสินค้าจะไม่ถูกเปลี่ยนแปลงหากผู้ใช้เข้าไปแก้ไขโปรไฟล์ที่อยู่ในตารางผู้ใช้ในอนาคต

ฟังก์ชันที่ 8: การอัปเดตสถานะคำสั่งซื้อแบบกลุ่ม (Admin Bulk Update Status)

คุณสมบัตินี้ได้รับการออกแบบมาเพื่อเสริมสร้างประสิทธิภาพในการปฏิบัติงานของแอดมิน โดยลดภาระการทำงานซ้ำซ้อน (Repetitive tasks) แอดมินสามารถทำเครื่องหมาย (Checkbox) เลือกผู้ใช้และคำสั่งซื้อได้หลายรายการ และส่งคำสั่งเปลี่ยนสถานะเป็น "ยืนยันคำสั่งซื้อ" (Bulk Update) ในคลิกเดียว [8]

กระบวนการทางลอจิก:

1. หน้า HTML Table จะมีคอลัมน์ของ <input type="checkbox" name="selected_orders" value="OrderNumber">[33]
2. เมื่อแบบฟอร์มถูกส่ง (Submit) PHP จะรับตัวแปรอาร์เรย์ของ selected_orders
3. ใช้ฟังก์ชัน implode("','", $selected_orders) ของ PHP เพื่อแปลงอาร์เรย์เป็นชุดตัวอักษร
4. ประมวลผลคำสั่ง SQL เพียงคำสั่งเดียวเพื่ออัปเดตข้อมูลทั้งหมดพร้อมกัน:
   UPDATE Orders SET status_id = 2 WHERE order_number IN ('ORD-001', 'ORD-002')
   เทคนิคนี้ช่วยลดจำนวนรอบการเชื่อมต่อระหว่าง Web Server และ Database ทำให้ระบบโดยรวมมี

ประสิทธิภาพสูงสุดเมื่อทำงานในระดับสเกล (Scalable approach)

5. การออกแบบส่วนติดต่อผู้ใช้และประสบการณ์ผู้ใช้ (UX/UI Design Specifications)

เพื่อให้การพัฒนาระบบมีความแม่นยำสูงตาม "ภาพตัวอย่าง" ที่ให้มา ส่วนนี้จะทำการแจกแจงข้อกำหนดด้าน UI อย่างละเอียดในทุกจุดสัมผัส (Touchpoints) ของระบบ โดยมุ่งเน้นไปที่หน้าจอ **รายการสินค้าและการจัดการตะกร้า (Product Listing & Cart Interface)** [35]

5.1 ภาพรวมของเค้าโครงหน้าจอ (Global Layout & Navigation)

หน้าจอถูกออกแบบด้วยสไตล์ความเรียบง่าย โปร่งสบาย และใช้พื้นที่ว่าง (Whitespace) อย่างมีประสิทธิภาพตามหลักการออกแบบ Dashboard ที่ดี [8]

- **แถบการนำทาง (Top Navigation Bar):** มุมซ้ายบนประกอบด้วยโลโก้แบรนด์ไอคอนสี่เหลี่ยมสีน้ำเงินพร้อมตัวอักษร **"OMS"** ถัดมาเป็นเมนูแบบแท็บ (Tab Navigation) สองสถานะ:
  - แท็บ **"สินค้า" (Products)** อยู่ในสถานะถูกเลือก (Active) มีพื้นหลังสีฟ้าอ่อนและตัวอักษรสีน้ำเงินเข้ม พร้อมไอคอนร้านค้า
  - แท็บ **"คำสั่งซื้อของฉัน" (My Orders)** อยู่ในสถานะไม่ถูกเลือก มีไอคอนรูปรายการ
- **แถบโปรไฟล์ผู้ใช้ (User Profile Bar):** มุมขวาบนแสดงไอคอนรูปคน พร้อมชื่อผู้ใช้งาน "สมชาย ใจดี" (จำลองการ Login สำเร็จ) และปุ่ม "ออกจากระบบ" (Logout) ที่มีไอคอนลูกศรชี้ออก

5.2 พื้นที่ส่วนหัวของเนื้อหา (Content Header & Action Bar)

- **ชื่อหน้า (Page Title):** แสดงข้อความ **"รายการสินค้า"** ด้วยตัวอักษรขนาดใหญ่ (H1) พร้อมข้อความกำกับย่อยด้านล่างว่า "เลือกสินค้าแล้วกด 'สร้าง Order'"
- **กล่องเครื่องมือเชิงบริบท (Contextual Tools):** ด้านขวาของพื้นที่ส่วนหัว มีองค์ประกอบสำคัญสองส่วน:
  1. ปุ่มสีน้ำเงิน **"สร้าง Order (1)"** ซึ่งมีไอคอนรูปกล่องและแสดงตัวเลข '1' อยู่ในวงกลมสีขาว แสดงถึงจำนวนรายการสินค้า (ชนิดสินค้า) ที่ถูกเลือกไว้แล้ว [9]
  2. ช่องป้อนข้อมูลสำหรับค้นหา (Search Input) มีไอคอนแว่นขยายและข้อความ Placeholder ว่า **"ค้นหาสินค้า..."**

5.3 แถบสรุปรายการคำสั่งซื้อ (Cart Summary Banner)

เมื่อผู้ใช้เริ่มปฏิสัมพันธ์กับสินค้า ระบบจะแสดงแถบสรุปที่มีพื้นหลังสีฟ้าอ่อน เพื่อเป็นจุดรวมสายตา (Visual Hierarchy) และให้ความชัดเจนถึงสถานะการทำงานปัจจุบัน

- ข้อความแสดงผล: **"เลือกไว้ 1 ชิ้น - ยอดรวม ฿18,900"** (จำนวนชิ้นและยอดรวมจะต้องสอดคล้องกับตรรกะคณิตศาสตร์ของการเลือกสินค้าด้านล่าง)
- ปุ่มการกระทำหลัก (Primary Call-to-Action): ปุ่มสีน้ำเงินเข้มข้อความ **"ดำเนินการสั่งซื้อ →"** ซึ่งจะทำหน้าที่แปลงข้อมูลบนหน้าเว็บเข้าสู่กระบวนการของ "ฟังก์ชันที่ 5: การเพิ่มคำสั่งซื้อสินค้า" ต่อไป

5.4 โครงสร้างกริดและการ์ดสินค้า (Product Grid & Card Anatomy)

การแสดงผลแคตตาล็อกสินค้าใช้การจัดวางแบบกริด (Grid Layout) แบบ 4 คอลัมน์ (4 Cards per row) เพื่อ

เพิ่มประสิทธิภาพการใช้พื้นที่บนหน้าจอเดสก์ท็อป การ์ดแต่ละใบมีกรอบเส้นขอบสีเทาอ่อนและขอบโค้งมน (Rounded Corners)

องค์ประกอบภายในการ์ดสินค้าแต่ละใบประกอบด้วย:

1. **ป้ายกำกับ SKU (SKU Badge):** แสดงอยู่ที่มุมซ้ายบนของบัตร ด้วยตัวอักษรขนาดเล็กในกรอบสีเทา เช่น SKU-001
2. **ชื่อสินค้า (Product Name):** ข้อความตัวหนาขนาดกลาง เช่น "แล็บท็อป Asus VivoBook 15"
3. **ราคาสินค้า (Price):** แสดงด้วยตัวอักษรสีน้ำเงินและมีขนาดใหญ่ที่สุดในการ์ด เพื่อเน้นย้ำความสำคัญทางธุรกิจ เช่น **"฿18,900"**
4. **ปริมาณสินค้าคงเหลือ (Stock Availability):** ข้อความสีเทาขนาดเล็กใต้ราคา แสดงจำนวนสต๊อก เช่น "คงเหลือ 13 ชิ้น"

สถานะของปุ่มควบคุมสินค้า (Button States & Interactions): ภาพตัวอย่างได้แสดงพฤติกรรมการโต้ตอบและสถานะที่แตกต่างกัน 3 รูปแบบอย่างชัดเจน ซึ่งเป็น Best Practice ในด้าน E-commerce UX [34]:

- **สถานะปกติ (Default State):** สินค้าที่มีในสต๊อกและยังไม่ถูกเลือก (เช่น SKU-002, 003, 004, 005, 007, 008) จะแสดงปุ่มสีน้ำเงินทึบขนาดเต็มความกว้างของการ์ด มีเครื่องหมายบวกและข้อความ **"+ เพิ่มในตะกร้า"**
- **สถานะสินค้าหมด (Out of Stock State):** สำหรับสินค้าที่ไม่มีในคลัง (เช่น SKU-006 เว็บแคม Logitech C920 คงเหลือ 0 ชิ้น) จะมีข้อความแจ้งเตือนตัวอักษรสีแดง **"สินค้าหมด"** อยู่ที่มุมขวาบนของการ์ด และปุ่มเพิ่มลงตะกร้าจะถูกลดความเข้มลง (Faded/Disabled) เพื่อป้องกันการกดสั่งซื้อ
- **สถานะถูกเลือก (Selected/Active State):** เมื่อผู้ใช้กดเพิ่มสินค้าลงตะกร้าแล้ว (เช่น SKU-001) พื้นที่ของปุ่มจะเปลี่ยนรูปลักษณ์เป็นตัวควบคุมจำนวนสินค้า (Quantity Stepper) ที่มีพื้นหลังสีขาว กรอบสีเทา โดยมีปุ่ม **[-]** ทางซ้าย ตัวเลขจำนวน **** ตรงกลาง และปุ่ม **[+]** ทางขวา กลไกนี้ช่วยให้ผู้ใช้สามารถปรับเพิ่มหรือลดจำนวนสินค้าได้ทันทีโดยไม่ต้องเปลี่ยนหน้าต่าง

5.5 การออกแบบ UI สำหรับผู้ดูแลระบบ (Admin Bulk Action UI)

แม้ในภาพตัวอย่างจะไม่มีหน้าจอ Admin ให้เห็นโดยตรง แต่จากข้อกำหนดการทำงานฟังก์ชันที่ 8 (Bulk Update) การออกแบบอินเทอร์เฟซในส่วนนี้ต้องเชื่อมโยงหลักการ UX ขั้นสูง [9]

- ตารางรายการคำสั่งซื้อของแอดมินต้องมีคอลัมน์แรกเป็น Checkbox การแสดง Checkbox จำเป็นต้องสอดคล้องกับมาตรฐานคือ ขนาดคลิกได้อย่างน้อย 24x24 px เพื่อลดข้อผิดพลาดในการคลิก (Fitts's Law) [34]
- เมื่อแอดมินคลิกที่ Checkbox ของหัวตาราง (Master Checkbox) จะเป็นการทำงานคำสั่ง Select All ทุกแถวในหน้านั้น
- แถวข้อมูล (Row) ใดที่ถูกเลือกด้วย Checkbox พื้นหลังของแถวนั้นจะเปลี่ยนสีไฮไลท์อ่อนๆ เพื่อให้แอดมินมองเห็นภาพรวมชัดเจนก่อนการอัปเดตสถานะแบบกลุ่ม ลดความเสี่ยงของการเผลออนุมัติคำสั่งซื้อที่ผิดพลาด [9]

6. บทสรุปและทิศทางในอนาคต (Conclusion & Future Outlook)

เอกสารข้อกำหนดความต้องการของระบบ (PRD) ฉบับนี้ ได้สร้างพิมพ์เขียวที่ครอบคลุมทุกมิติของการพัฒนา

ระบบ Order Management System ด้วยเทคโนโลยี PHP 7 รูปแบบ Monolithic Server-Side สำหรับธุรกิจอีคอมเมิร์ซขนาดเล็ก โครงสร้างที่ถูกวิเคราะห์ไว้ได้ตอบสนองต่อโจทย์ความต้องการครบถ้วนทั้ง 8 ข้อ ตั้งแต่กลไกการสร้างข้อมูลพื้นฐาน (Auto Seed) ไปจนถึงการยกระดับประสิทธิภาพการทำงานของแอดมินด้วยฟังก์ชันประมวลผลคำสั่งซื้อแบบกลุ่ม (Bulk Actions) [8] และการออกแบบโครงสร้างฐานข้อมูลเชิงสัมพันธ์ที่สามารถรองรับความสัมพันธ์แบบ 1-to-Many ของคำสั่งซื้อและรายการสินค้าได้อย่างเป็นระบบ [16]

การถอดรหัส UI จากภาพตัวอย่างและแปลความไปสู่ระบบ UX/UI ที่จับต้องได้ แสดงให้เห็นถึงความใส่ใจในบริบทของผู้บริโภค ไม่ว่าจะเป็นกลไกการโต้ตอบของปุ่มรถเข็น (Cart Stepper) หรือการแจ้งเตือนสภาวะที่สินค้าหมดสต๊อก สิ่งเหล่านี้จะส่งเสริมให้แพลตฟอร์มนี้เป็นเครื่องมือที่มีประสิทธิภาพในการผลักดันยอดขาย ลดการทำงานที่ซ้ำซ้อน และเพิ่มความรวดเร็วในการบริการ

แม้กระนั้น ภายใต้ข้อจำกัดของโปรเจกต์ต้นแบบที่ตั้งใจ "ละเว้นประเด็นด้านความปลอดภัย" ทางทีมสถาปัตยกรรมยังคงต้องตระหนักถึงหนี้สินทางเทคนิค (Technical Debt) ที่เกิดขึ้น หากในอนาคตธุรกิจมีการเติบโตและขยายตัว จำเป็นอย่างยิ่งที่จะต้องพิจารณานำมาตรฐานความปลอดภัยเข้าสู่ระบบ ทั้งในส่วนของการเข้ารหัสรหัสผ่าน (Password Hashing) และการป้องกันการฉีดโค้ด SQL (SQL Injection Prevention) รวมถึงอาจพิจารณาการแยกบริการ (Decoupling) ในส่วนของระบบสินค้าและคำสั่งซื้อออกจากกัน เพื่อปูทางสู่สถาปัตยกรรมระดับ Microservices ต่อไปในระยะยาว [11] โครงสร้างไดเรกทอรีและการแบ่งตารางข้อมูลที่นำเสนอในเอกสารฉบับนี้ ได้เตรียมพร้อมสำหรับกระบวนการปรับโครงสร้าง (Refactoring) ดังกล่าวไว้ล่วงหน้าอย่างดีเยี่ยมแล้ว

ผลงานที่อ้างอิง

1. Thailand E-commerce Market Guide: Key Trends & Insights for 2026 - Anchanto, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://anchanto.com/thailand-e-commerce-industry/
2. What is Order Management System | Bluestone PIM Glossary, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.bluestonepim.com/glossary/order-management-system
3. Online Order Management System (OMS) - Can Innovation, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.caninnovation.co.th/en/oms-en/
4. What is a Product Requirements Document? From Static PRD to Dynamic Strategy, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.productboard.com/blog/product-requirements-document-guide/
5. What is a Product Requirements Document? - Visure Solutions, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://visuresolutions.com/alm-guide/product-requirements-document/
6. Monolithic Architecture Guide: Pros, Cons & Evolution Paths - Strapi, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://strapi.io/blog/monolithic-architecture-pros-cons-evolution-guide
7. From Monolithic to Microservices: Architectures 101 - DEV Community, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://dev.to/encore/from-monolithic-to-microservices-architectures-101-3f2e
8. Admin Dashboard UI/UX: Best Practices for 2025 | by Carlos Smith | Medium, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://medium.com/@CarlosSmith24/admin-dashboard-ui-ux-best-practices-for-2025-8bdc6090c57d
9. Bulk Actions: 3 Design Guidelines - YouTube, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026

https://www.youtube.com/watch?v=7MjvBqlRzCY
10. PHP Monolith to Microservices: When to Split Web Applications | Zend, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.zend.com/blog/php-monolith-to-microservices
11. Microservices vs. Monolith: A PHP Developer's Guide to Architecture Decisions in 2025, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.fuseweb.nl/en/blog/2025/02/03/microservices-vs-monolith-php-architecture-guide-2025
12. On structuring PHP projects - Nikola Poša, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.nikolaposa.in.rs/blog/2017/01/16/on-structuring-php-projects/
13. PHP: The Right Way | Reference for PHP best practices, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://phptherightway.com/
14. Building Scalable PHP Applications: Best Practices for Performance and Security, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://dev.to/olatejuthedev/building-scalable-php-applications-best-practices-for-performance-and-security-56jj
15. Designing the database schema for a new e-commerce platform and considering factors like normalization, denormalization, indexing, and performance optimization to ensure efficient storage and retrieval of data. | by Biswanath Giri | Medium, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://bgiriaicloud.medium.com/designing-the-database-schema-for-a-new-e-commerce-platform-and-considering-factors-like-ec28d4fb81db
16. Database Project: E-commerce Order Management System with SQL | by Karlos | Medium, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://medium.com/@karlos-b/database-project-e-commerce-order-management-system-with-sql-d986b044d92
17. How to Design a Relational Database for E-commerce Website - GeeksforGeeks, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.geeksforgeeks.org/dbms/how-to-design-a-relational-database-for-e-commerce-website/
18. eCommerce Database Design - DEV Community, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://dev.to/ezzdinatef/ecommerce-database-design-1ggc
19. Designing a Database Schema for Order Workflow - YouTube, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.youtube.com/watch?v=5cZ9ri-Dfds
20. Ecommerce Database Design: ER Diagram for Online Shopping - Redgate Software, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.red-gate.com/blog/er-diagram-for-online-shop/
21. High-Performance Order Storage: Database Schema - The WooCommerce Developer Blog, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://developer.woocommerce.com/2022/09/15/high-performance-order-storage-database-schema/
22. Database design for Order Management - sql server - Stack Overflow, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://stackoverflow.com/questions/22346276/database-design-for-order-management
23. Extreme UltraDev - E-commerce Database Design Part 1, เข้าถึงเมื่อ กุมภาพันธ์ 22,

2026 https://www.princeton.edu/~rcurtis/ultradev/ecommdatabase.html
24. 10 Efficient (and Fun) Ways to Seed Your Database - Tighten Co., เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://tighten.com/insights/10-efficient-and-fun-ways-to-seed-your-database/
25. How I use database seed scripts for local development - YouTube, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.youtube.com/watch?v=Qa29-h1xLeg
26. How to check if table is empty - Databases - SitePoint Forums, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.sitepoint.com/community/t/how-to-check-if-table-is-empty/1311
27. How to check if database is empty? - php - Stack Overflow, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://stackoverflow.com/questions/65626121/how-to-check-if-database-is-empty
28. 5 Registration Form Usability Guidelines, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://usabilitygeek.com/5-registration-form-usability-guidelines/
29. Browse thousands of Order List Dashboard images for design inspiration - Dribbble, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://dribbble.com/search/order-list-dashboard
30. Best structure for "Orders" table having multiple Items per order number - Stack Overflow, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://stackoverflow.com/questions/2000833/best-structure-for-orders-table-having-multiple-items-per-order-number
31. What Are the Benefits of Order Management Database Structure? - fabric Inc., เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://fabric.inc/blog/product/order-management-database-structure
32. Bulk action UX: 8 design guidelines with examples for SaaS - Eleken, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.eleken.co/blog-posts/bulk-actions-ux
33. Bulk Update - htmx ~ Examples, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://htmx.org/examples/bulk-update/
34. Checkbox UI design: Best practices and examples - LogRocket Blog, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://blog.logrocket.com/ux-design/checkbox-ui-design-best-practices-examples/
35. Dashboard Design UX Patterns Best Practices - Pencil & Paper, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.pencilandpaper.io/articles/ux-pattern-analysis-data-dashboards
36. 20 Best Practices & Examples for Better Dashboard Designs - Mockplus, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://www.mockplus.com/blog/post/dashboard-design-best-practices-examples
37. Case study: Designing for status changes | by UX Monster | Bootcamp | Medium, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026 https://medium.com/design-bootcamp/ux-design-for-status-19e8a92b2aa3
38. Decoupling a monolithic PHP application: a practical example - Lokalise Blog, เข้าถึงเมื่อ กุมภาพันธ์ 22, 2026

https://lokalise.com/blog/decoupling-a-monolithic-php-application-a-practical-example/